

# HackBridge

*Concept Document — Multi-Stakeholder Hackathon Platform*

## 1. Project Title

HackBridge: A White-Label, Multi-Tenant Hackathon Management Platform for Engineering Colleges

## 2. Pilot Institution

Maharaja Institute of Technology Thandavapura (MITT), Thandavapura, Mandya, Karnataka

## 3. Background and Motivation

Engineering colleges across Karnataka regularly organize hackathons and industry-linked innovation events, but the process is typically fragmented — problem statements are collected over email, team formation happens on WhatsApp, submissions are scattered across spreadsheets and drive links, and judging relies on paper rubrics or ad hoc scoring sheets. This creates avoidable overhead for organizing committees, poor visibility for industry partners, and no durable, portfolio-grade record for participating students.

HackBridge is conceived as a single platform that brings every stakeholder in a hackathon lifecycle — the college, industry problem-setters, students, and judges — onto one coordinated system, while being built from day one as multi-tenant infrastructure so that a working deployment at one institution (MITT) can be extended to other colleges as independent, brand-isolated tenants without re-architecting the system.

## 4. Vision

To provide a reusable, secure, white-label SaaS foundation that any engineering college can adopt to run industry-integrated hackathons — reducing administrative overhead for institutions, giving companies a structured channel to source real problems and real talent, and giving students a verifiable, portfolio-building record of their participation and performance.

## 5. Core Problem Statement

There is no purpose-built, secure, multi-tenant platform that simultaneously serves institutions that need to run and govern hackathon lifecycles, companies that want to submit real-world problem statements and scout talent, students who need a structured way to form teams and submit work, and evaluators who need a fair, conflict-of-interest-aware scoring mechanism — all within a single system that can be safely white-labelled and rolled out to multiple colleges without data leaking across institutional boundaries.

## 6. Target Stakeholders

| Stakeholder                                 | Role in the Platform                                                                                                                                                                   |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>College Admin &amp; Committee</b>        | Own the hackathon lifecycle end-to-end: event creation, state-machine driven stage transitions, rubric configuration, conflict-of-interest checks, and real-time analytics dashboards. |
| <b>Industry Problem Setters (Companies)</b> | Submit structured challenge statements, provision datasets, and use the platform as a talent pipeline / scouting channel.                                                              |

|                                |                                                                                                                                                                                               |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Student Innovators</b>      | Form teams via invite codes, make multi-format submissions (abstract, GitHub repository, live demo link), and build a verified talent portfolio over time.                                    |
| <b>Judges &amp; Evaluators</b> | Score submissions against normalized, weighted rubric criteria, with automatic conflict-of-interest filtering so evaluators are not matched against teams they have a declared conflict with. |
| <b>Super Admin</b>             | Platform-level oversight across all tenant institutions (cross-college administration).                                                                                                       |
| <b>Mentor</b>                  | Supports student teams during the hackathon lifecycle. Role is defined in the authorization model; the dedicated mentor workflow is planned for a later phase.                                |

The `user_role` enum formalizes the seven stakeholder/administrative roles at the database level.

## 7. Differentiating Design Principles

### 7a. Multi-tenancy as a first-class concern, not an afterthought

HackBridge is designed so that every institution operates as an isolated tenant. Isolation is enforced natively at the database layer using PostgreSQL Row Level Security (RLS) and security-definer helper functions (`get_current_user_tenant_id()`, `is_college_admin()`, `is_super_admin()`), not merely by application-level filtering — so a bug in the frontend cannot leak one college's data into another's.

### 7b. White-labelling by design

White-labelling is established as an architectural capability through tenant-specific configuration, branding settings, and subdomain/custom-domain fields, injected at runtime as CSS variables (`--brand-primary`, `--brand-secondary`). Full production tenant onboarding and multi-college deployment are planned for later phases.

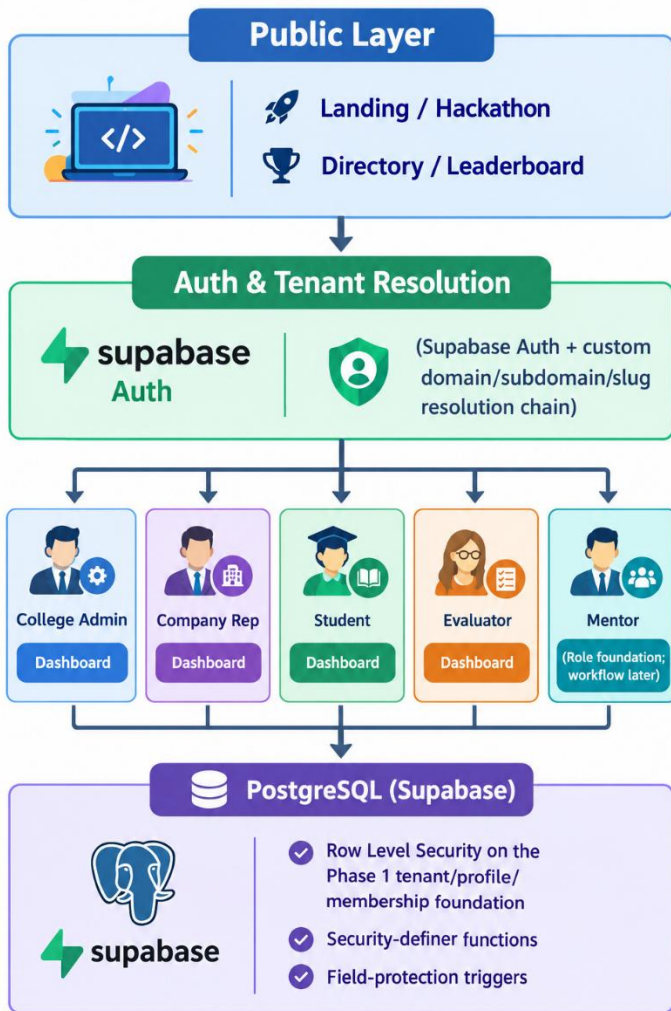
### 7c. Security-first identity handling

Role and tenant assignment are never trusted from the client. At sign-up, the server-side `handle_new_user()` function accepts only self-service roles (student, company representative, evaluator, mentor), validates any client-supplied tenant against an active tenant's email-domain policy, and otherwise resolves the tenant from the user's verified email — falling back to the pilot tenant only when nothing else applies. A dedicated database trigger protects sensitive identity and authorization fields from unauthorized self-modification, including role and tenant assignment, independently of the frontend.

### 7d. Fail-closed behaviour

Protected workspaces are designed to fail closed when required backend configuration is unavailable. Demonstration/mock content is used during the current prototype stage and will be replaced by real database-driven workflows in subsequent phases.

## 8. Proposed System Architecture



## 9. Technology Stack

| Layer                                  | Technology                                                                                                                                   |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Frontend</b>                        | React + TypeScript, Vite build tooling, Tailwind CSS                                                                                         |
| <b>Auth &amp; Backend-as-a-Service</b> | Supabase (Auth, Postgres, Row Level Security)                                                                                                |
| <b>Styling / White-labelling</b>       | Tailwind + CSS custom properties for brand injection                                                                                         |
| <b>Planned (Phase 2+)</b>              | Redis/BullMQ (queues), object/file storage (Supabase Storage or S3-compatible storage), Express/tRPC (custom API layer), Docker (deployment) |

## 10. Current Implementation Status — Phase 1 & Phase 1.5

### Phase 1 — Foundation — Completed

The first implementation phase has established the working technical foundation of HackBridge:

- React + TypeScript + Vite frontend
- Tailwind-based UI
- Supabase authentication integration
- MITT pilot tenant configuration
- Tenant, profile and membership database foundation
- Seven-role authorization model
- Role-based application routing
- Supabase Row Level Security for Phase 1 tenant/profile/membership data
- Public landing, registration and login interfaces
- Student, Company and Evaluator authentication flows
- Student workspace and stakeholder dashboard shells
- Session persistence and logout handling
- Tenant-aware application configuration
- Successful end-to-end student registration and login validation

## Phase 1.5 — Security Hardening — Completed

The security hardening phase added:

- Database-level protection of sensitive profile and authorization fields
- Protection against unauthorized self-modification of role and tenant assignment
- Database-side validation of account creation metadata
- Tenant-aware user creation logic
- MITT pilot tenant fallback configuration
- Explicit RLS policies and security-definer helper functions
- Production-oriented fail-closed behavior for protected application areas

## Current Limitation

The current stakeholder dashboards are UI shells with representative/mock data where the corresponding business workflows have not yet been implemented.

The following major application capabilities remain pending:

- Hackathon creation and lifecycle management
- Company and problem-statement management
- Team formation and invite codes
- Student problem selection
- Real project submissions and file storage
- Evaluator assignment and scoring
- Conflict-of-interest workflow
- Score aggregation and production leaderboard
- Talent profiles and talent pool
- Company hiring workflow
- AI-assisted screening
- Notifications
- Persistent audit logging
- Operational analytics

- Background job infrastructure
- Production storage infrastructure
- Multi-college production deployment

## 11. Security Hardening Highlights (Phase 1.5)

- Profile privilege escalation is blocked in the database: a BEFORE INSERT OR UPDATE trigger on public.profiles refuses any change a non-super-admin makes to sensitive identity and authorization fields, and refuses to move a profile into or out of a tenant, while self-service editing of non-sensitive fields (full name, phone, avatar, metadata) still works.
- Role and tenant assignment during account creation are resolved and validated by database-side signup logic rather than being trusted directly from client input.
- No placeholder tenant identifiers: the frontend sends a tenant ID only when it comes from a real tenant row; if resolution fails, the field is omitted and the database resolves it server-side.
- Protected routes are designed to fail closed: production builds render a configuration error screen when required backend configuration is unavailable, rather than falling back to a permissive state.
- For the MITT pilot demonstration, no college email domain has been configured, allowing the tenant-resolution fallback path to be exercised without relying on a domain-specific email policy.

## 12. Repository & Deployment Notes

Database migrations are version-controlled and currently applied manually through the Supabase SQL Editor in the prescribed order. Migration dependencies are documented to prevent accidental reversion of security-hardening changes. Detailed migration sequencing and operational cautions are maintained separately in the project's technical implementation notes.

Frontend configuration is via environment variables (VITE\_SUPABASE\_URL, VITE\_SUPABASE\_ANON\_KEY), with a template .env.example file provided for local setup.

## 13. Proposed Implementation Roadmap

| Phase                                               | Status    | Scope                                                                                                                                                  |
|-----------------------------------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Phase 1 — Foundation</b>                         | Completed | Multi-tenant schema, Supabase Auth, RLS on the tenant/profile/membership foundation, role-based routing, white-labelling foundation, dashboard shells. |
| <b>Phase 1.5 — Security Hardening</b>               | Completed | Database-level protection of sensitive profile fields, server-side role/tenant resolution at sign-up, fail-closed protected routes.                    |
| <b>Phase 2 — Hackathon &amp; Problem Management</b> | Planned   | Hackathon creation, lifecycle/state-machine transitions, company registration, problem-statement intake and review.                                    |
| <b>Phase 3 — Teams &amp; Submissions</b>            | Planned   | Team creation, invite codes, problem selection, draft and final submissions, file/link handling.                                                       |
| <b>Phase 4 — Evaluation &amp; Results</b>           | Planned   | Evaluator assignment, rubric configuration, scoring, conflict-of-interest workflow, score aggregation, leaderboard.                                    |

|                                            |         |                                                                                                                                                                     |
|--------------------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Phase 5 — Talent &amp; Intelligence</b> | Planned | Verified talent profiles, company-side talent discovery, hiring workflow, AI-assisted pre-screening.                                                                |
| <b>Phase 6 — Platform Maturity</b>         | Planned | Notifications, persistent audit logs, storage infrastructure, background job processing, operational analytics, production deployment and multi-college onboarding. |

## 14. Expected Outcomes

These are anticipated benefits once the roadmap above is delivered, not claims about the current state of the project:

- A single, secure system of record for hackathons at MITT, replacing email- and spreadsheet-driven coordination.
- A reusable, white-label foundation that can onboard additional engineering colleges as isolated tenants with minimal re-engineering.
- A structured channel for industry partners to submit real problems and discover student talent.
- A verifiable, portfolio-grade participation and performance record for students across multiple hackathon cycles.
- A security posture — database-enforced tenant isolation, server-side identity resolution, and fail-closed routing — suitable for handling multi-institutional data safely as functionality is built out.

*This concept document reflects the HackBridge project's design intent and its verified Phase 1 / Phase 1.5 implementation status. It deliberately distinguishes the architectural target from what is currently built and functional, so it can be used safely in project review.*